

Algoritmi-Modulo algoritmi per bioinformatica

Heapsort-Capitolo 6 [CLRS]

Roberto Posenato

versione 2026-2, 13/04/2026

- 1 Heapsort
 - Idea generale
 - Struttura dati heap
 - Algoritmo
- 2 Code di priorità
 - Operazioni fondamentali

*“Tell me and I forget, teach me and I remember,
involve me and I learn.”*

Benjamin Franklin

*“A student who achieves knowledge through free investigation
and spontaneous effort will be able to retain that knowledge
and will have acquired a methodology that can serve for a life-
time.”*

Jean Piaget

Scopo della lezione

- Introdurre l'ultimo algoritmo di ordinamento per confronti:
HEAPSORT

Nota!

In questa lezione consideriamo il problema dell'ordinamento d'interi senza perdere di generalità.

I risultati valgono per qualsiasi array di valori confrontabili fra loro mediante una relazione d'ordine.

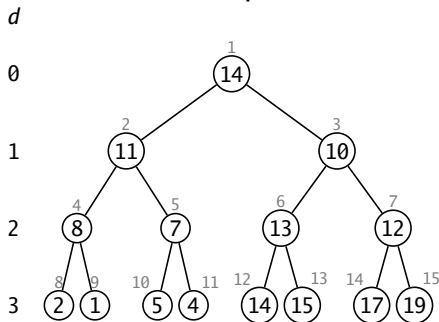
HEAPSORT

- Ordina sul posto (come INSERTIONSORT) e ha complessità $O(n \lg n)$ (come MERGESORT).
- Nel caso medio, è meno veloce di QUICKSORT perché le costanti nascoste in $O(n \lg n)$ sono più piccole in QUICKSORT che in HEAPSORT.
- È un esempio di un'altra tecnica di progettazione algoritmica:
 - è basato sulla struttura dati **heap**
 - **heap** è usata anche in altri algoritmi importanti come, per esempio, algoritmi per le code di priorità.

HEAPSORT

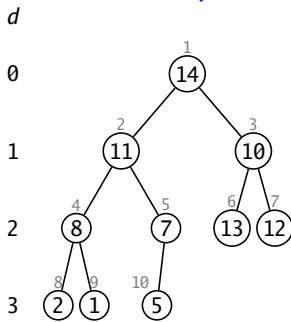
Heap: alberi binari e alberi binari quasi completi

Albero binario completo



Ciascun nodo interno ha esattamente due nodi figli. Si può anche dire che le foglie sono tutte allo stesso livello.

Albero binario quasi completo



Tutti i livelli devono essere completamente riempiti tranne eventualmente l'ultimo. L'ultimo livello è sempre riempito da sinistra verso destra. (Se si numerano i nodi per livelli, la numerazione non deve avere "buchi").

HEAPSORT

Heap

Definizione 1 (Heap (binario))

Uno **heap** è un array che rappresenta un *albero binario quasi completo*:

- ogni nodo dell'albero corrisponde a un elemento dell'array;
- tutti i livelli dell'albero devono essere completamente riempiti tranne eventualmente l'ultimo che può essere parziale (ma partendo comunque da sinistra);
- Se A è uno heap,
 $A.length = \#$ elementi dell'array,
 $A.heapSize = \#$ nodi dell'albero rappresentati:
 $0 \leq A.heapSize \leq A.length$;
- la radice dell'albero è in $A[1]$.

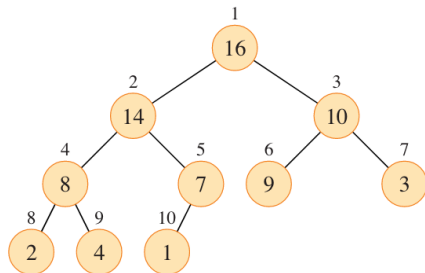
HEAPSORT

Heap

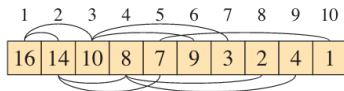
Definizione 1 (continua)

- Se i è l' i -esimo nodo, gli indici del nodo genitore e dei due nodi figli sono dati da:
 - **PARENT(i):** return $\lfloor i/2 \rfloor$;
 - **LEFT(i):** return $2i$;
 - **RIGHT(i):** return $2i + 1$;

Nota: moltiplicare/dividere per 2 richiede solo di spostare a sx/dx 1 bit!



(a)



(b)

HEAPSORT

Heap

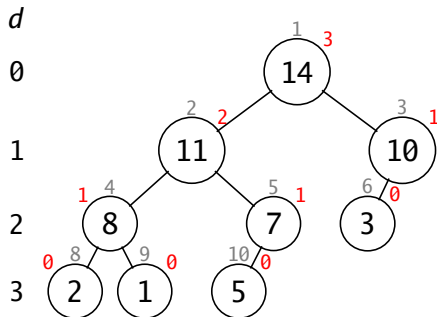
- Due tipi di heap binari:
 - **max-heap**: $A[\text{PARENT}(i)] \geq A[i]$ per ogni $i \neq 1$.
 - **min-heap**: $A[\text{PARENT}(i)] \leq A[i]$ per ogni $i \neq 1$.
- Algoritmo HEAPSORT usa i max-heap.
- I min-heap sono usati per realizzare le code di priorità.

HEAPSORT

Heap: alberi binari e alberi binari quasi completi

Definizione 2

- **Altezza di un nodo** è il # di archi del cammino semplice più lungo che parte dal nodo e termina in una foglia.
(# livelli sottostanti al livello dove si trova il nodo)
- **Altezza dello heap** = altezza della radice.



In rosso l'altezza di ciascun nodo.

HEAPSORT

Heap: alberi binari e alberi binari quasi completi

- Un albero binario completo di altezza h ha $n = 2^{h+1} - 1$ nodi.
Esempio: $h = 3 \Rightarrow n = 15$.
- Un albero binario quasi completo ha $2^h - 1 < n \leq 2^{h+1} - 1$ nodi.

- Un albero binario quasi completo di n nodi ha altezza $h = \lfloor \lg n \rfloor$.

Dimostrazione: Vale

$2^h \leq n < 2^{h+1}$. Applicando $\lg$,
 $h \leq \lg n < h + 1$, quindi $h = \lfloor \lg n \rfloor$.

Esempio:

$n = 10 \Rightarrow h = \lfloor \lg 10 \rfloor = \lfloor 3.32 \rfloor = 3$

n_d d h' ($h = 3$)

2^0 0 3

2^1 1 2

2^2 2 1

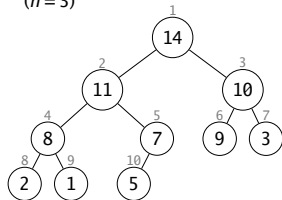
$\leq 2^3$ 3 0

$$\leq \sum_{k=0}^h 2^k$$

$$= \frac{2^{h+1}-1}{2-1} = 2^{h+1} - 1$$

Nota: serie geometrica finita

$$\sum_{k=0}^h 2^k = \frac{1-2^{h+1}}{1-2} = \frac{2^{h+1}-1}{2-1} = 2^{h+1} - 1$$



Quasi tutte le operazioni fondamentali sugli heap sono eseguite in un tempo proporzionale all'altezza dell'albero, quindi $O(\lg n)$.

HEAPSORT

Heap

Sono 6 le operazioni fondamentali relative a uno heap:

- ❶ BUILDMAXHEAP genera un max-heap da un array generico in tempo $O(n)$;
- ❷ MAXHEAPIFY ripristina la proprietà di max-heap quando a un elemento viene diminuito il valore. Tempo $O(\lg n)$ (operazione necessaria a BUILDMAXHEAP);
- ❸ MAXHEAPINSERT, ❹ HEAPEXTRACTMAX, ❺ HEAPINCREASEKEY e ❻ HEAPMAX consentono di usare uno heap per realizzare una coda di priorità. Hanno complessità $O(\lg n)$, tranne HEAPMAX che ha complessità $O(1)$.

HEAPSORT

Heap: MAXHEAPIFY

- Dato un max-heap $A[1 : n]$, se si modifica $A[i]$ **diminuendo il suo valore**, si deve garantire che A sia ancora un max-heap.
- Se $A[i]$ è più piccolo dei figli, si fa “scendere” $A[i]$ fino alla posizione corretta.
- A ogni passo, viene determinato il più grande fra $A[i]$, $A[\text{LEFT}(i)]$ e $A[\text{RIGHT}(i)]$;
- Se $A[i]$ è massimo, allora tutto è ok.
- Altrimenti viene scambiato con il figlio con valore maggiore.
- Si itera ricorsivamente.

Algoritmo: MAXHEAPIFY(A, i)

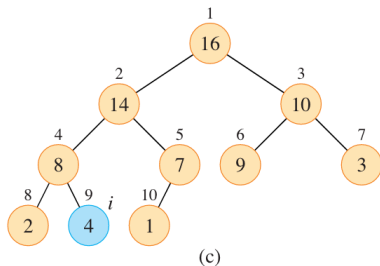
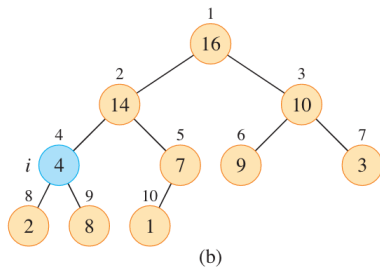
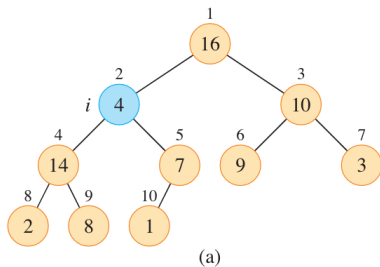
Input: A array la proprietà di max-heap che può essere violata solo nel sottoalbero radicato in i .

Output: A con la proprietà di max-heap ristabilita nel sottoalbero radicato in i .

```
1:  $l = \text{LEFT}(i)$ 
2:  $max = (l \leq A.heapSize \ \&\& \ A[l] > A[i]) ? l : i$ 
3:  $r = \text{RIGHT}(i)$ 
4: if ( $r \leq A.heapSize$  e  $A[r] > A[max]$ ) then
5:    $max = r$ 
6: if ( $max \neq i$ ) then
7:   Scambia  $A[i]$  con  $A[max]$ 
8:   MAXHEAPIFY( $A, max$ )
```

HEAPSORT

Heap: MAXHEAPIFY



HEAPSORT

Heap: MAXHEAPIFY: Analisi di complessità

- Sottoalbero da sistemare composto da n nodi (dimensione).
- $T(n) = T(\frac{n}{b}) + \Theta(1)$ in quanto c'è solo una chiamata ricorsiva e le altre operazioni interne sono indipendenti da n .
- Caso migliore: albero perfettamente bilanciato. $b = 2$;
- Caso peggiore: albero non è bilanciato e l'ultima riga dell'albero è piena a metà.
Il sottoalbero sx della radice ha un livello in più del sottoalbero dx.
Sottoalbero sx è quasi completo, mentre sottoalbero dx è completo.

HEAPSORT

Heap: MAXHEAPIFY: Analisi di complessità

- Nel caso peggiore sottoalbero sx contiene $< \frac{2}{3}n$ nodi, dove n è il numero totale dei nodi. $b = \frac{3}{2}$.

Dimostrazione: Se h è l'altezza della radice, il sottoalbero sx ha altezza $h - 1$ mentre il sottoalbero dx ha altezza $h - 2$, perché ha un livello in meno. Assumendo che il sottoalbero sx sia completo, il # totale di nodi è $n = 2^h - 1 + 2^{h-1} - 1 + 1 = 3 \cdot 2^{h-1} - 1$.

La proporzione del sottoalbero sx (che è quello che si deve esplorare) è

$$\frac{2^h - 1}{3 \cdot 2^{h-1} - 1} < \frac{2^h - 1}{3 \cdot 2^{h-1}} = \frac{2 \cdot 2^{h-1} - 1}{3 \cdot 2^{h-1}} = \frac{2}{3} - \frac{1}{3 \cdot 2^{h-1}} < \frac{2}{3}$$

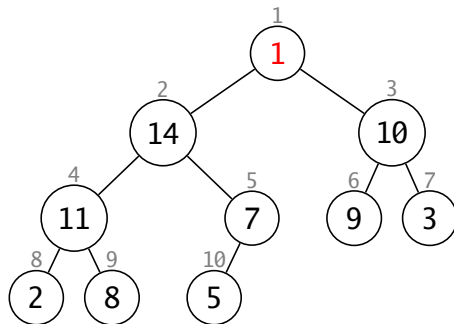
- $T(n) \leq T(\frac{2}{3}n) + \Theta(1) = O(\lg n)$ *
- La complessità di MAXHEAPIFY è quindi $T(n) = O(\lg n) = O(h)$.

*Si dimostra come caso 2 teorema esperto [CLRS]. L'equazione di ricorrenza $T(n) \leq T(\alpha n) + \Theta(1)$ ha soluzione $T(n) = O(\lg n)$ per qualsiasi $\alpha < 1$.

HEAPSORT

Heap: MAXHEAPIFY: Analisi di complessità: Esempio caso peggiore

Numero chiamate
ricorsive



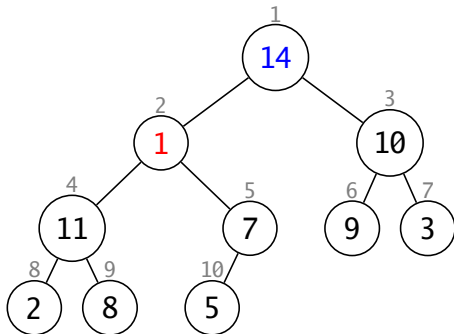
Nel caso peggiore, un elemento posto alla radice dell'albero (heap) deve scendere fino a una foglia. La complessità quindi è $\Omega(\lg n)$.

HEAPSORT

Heap: MAXHEAPIFY: Analisi di complessità: Esempio caso peggiore

Numero chiamate
ricorsive

1

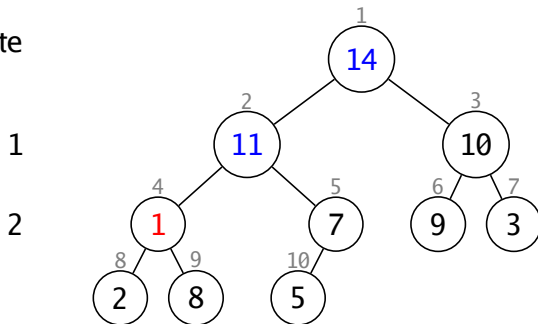


Nel caso peggiore, un elemento posto alla radice dell'albero (heap) deve scendere fino a una foglia. La complessità quindi è $\Omega(\lg n)$.

HEAPSORT

Heap: MAXHEAPIFY: Analisi di complessità: Esempio caso peggiore

Numero chiamate
ricorsive

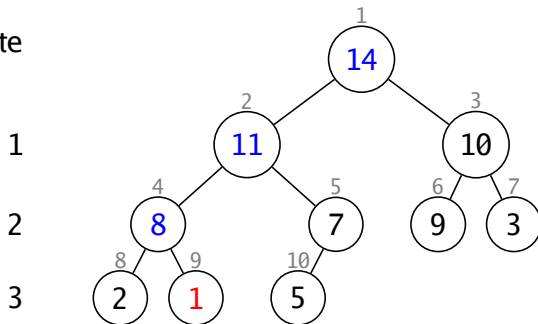


Nel caso peggiore, un elemento posto alla radice dell'albero (heap) deve scendere fino a una foglia. La complessità quindi è $\Omega(\lg n)$.

HEAPSORT

Heap: MAXHEAPIFY: Analisi di complessità: Esempio caso peggiore

Numero chiamate
ricorsive



Nel caso peggiore, un elemento posto alla radice dell'albero (heap) deve scendere fino a una foglia. La complessità quindi è $\Omega(\lg n)$.

HEAPSORT

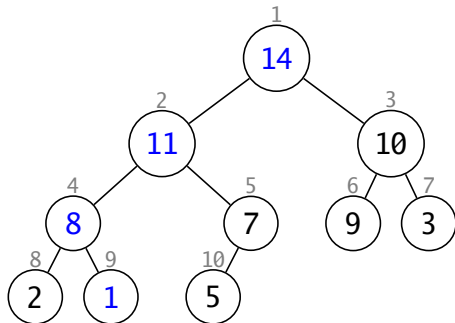
Heap: MAXHEAPIFY: Analisi di complessità: Esempio caso peggiore

Numero chiamate
ricorsive

1

2

$3 = \lfloor \lg 10 \rfloor$



Nel caso peggiore, un elemento posto alla radice dell'albero (heap) deve scendere fino a una foglia. La complessità quindi è $\Omega(\lg n)$.

HEAPSORT

Heap: Costruire un heap: BUILDMAXHEAP

- Dato un array di n elementi, si deve trasformarlo in uno heap.
- Tutti gli elementi in $A[\lfloor \frac{n}{2} \rfloor + 1 : n]$ sono foglie. Quindi ciascun elemento è già un heap. Si possono ignorare.
- A partire dalle foglie, si risale l'array ricollocando tutti gli elementi mediante MAXHEAPIFY.

Algoritmo: BUILDMAXHEAP(A)

Input: A array di n elementi

Output: A riordinato come max heap

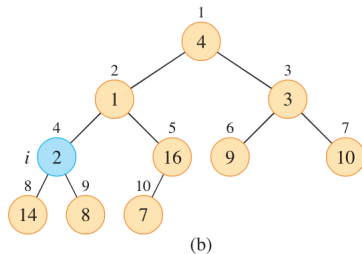
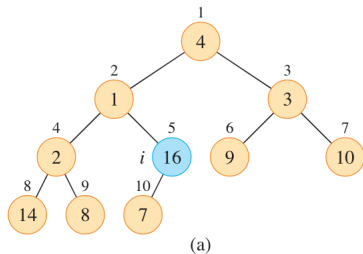
```
1:  $A.heapSize = A.length$   
2: for  $i = \lfloor A.length/2 \rfloor$  to 1 do  
3:    $\lfloor$  MAXHEAPIFY( $A, i$ )
```

HEAPSORT

Heap: Costruire un heap: BUILDMAXHEAP

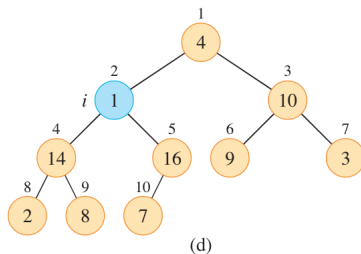
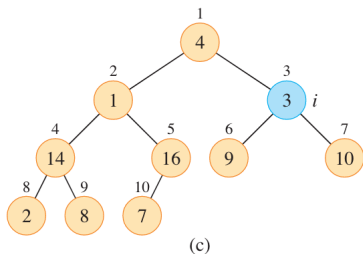
A

4	1	3	2	16	9	10	14	8	7
---	---	---	---	----	---	----	----	---	---



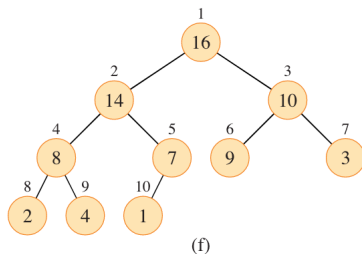
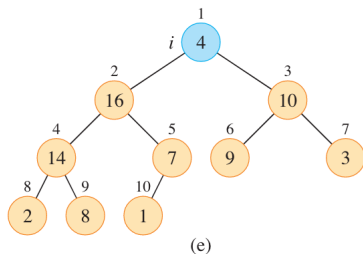
HEAPSORT

Heap: Costruire un heap: BUILDMAXHEAP



HEAPSORT

Heap: Costruire un heap: BUILDMAXHEAP



HEAPSORT

Heap: Costruire un heap: BUILDMAXHEAP: correttezza

Invariante di ciclo:

*All'inizio di ogni iterazione del ciclo **for**, ogni nodo d'indice $i + 1, \dots, n$ è la radice di un max heap.*

Inizializzazione:

Prima dell'inizio del I ciclo del **for**, $i = \lfloor n/2 \rfloor$. Quindi tutti i nodi successivi sono foglie e, quindi, radici di max heap di un solo elemento.

HEAPSORT

Heap: Costruire un heap: BUILDMAXHEAP: correttezza

Invariante di ciclo:

*All'inizio di ogni iterazione del ciclo **for**, ogni nodo $i + 1, \dots, n$ è la radice di un max heap.*

Conservazione:

- Si consideri il nodo alla posizione i . I figli hanno indice maggiore: sono radici di max heap per l'invariante.
- Se $A[i]$ non è maggiore dei figli, MAXHEAPIFY(A, i) modifica sottoalbero radicato in i , $A[i : n]$, rendendolo di nuovo max heap.
- Quindi all'inizio del nuovo ciclo ($i - 1$), i nodi $i, \dots, n$ sono radici di un max heap.

Conclusione:

- Alla fine del ciclo, $i = 0$, il nodo 1 è quindi la radice di un max heap che comprende tutti i nodi da 1 a n . A è un max heap.

HEAPSORT

Heap: Costruire un heap: BUILDMAXHEAP: complessità

- Considerare lo heap come albero facilita il calcolo della complessità computazionale.
- Per ogni nodo da sistemare, MAXHEAPIFY richiede al più un tempo proporzionale all'altezza del nodo.
- Caso peggiore: per ogni altezza possibile $0 \leq h' \leq h = \lfloor \lg n \rfloor$, MAXHEAPIFY richiede il tempo massimo per ciascun nodo di altezza h' .

All'altezza h' ci sono al più $n_{h'} = \lceil n/2^{h'+1} \rceil$ nodi.

Dimostrazione. Un heap di n nodi, contiene $\lceil n/2 \rceil$ foglie. Quindi $n_0 = \lceil n/2^{0+1} \rceil$. Supponiamo che valga $n_{h'-1} \leq \lceil n/2^{h'} \rceil$. Per induzione, dimostriamo $n_{h'} \leq \lceil n/2^{h'+1} \rceil$. Se $n_{h'-1}$ è pari, $n_{h'} = n_{h'-1}/2 = \lceil n_{h'-1}/2 \rceil$. Se $n_{h'-1}$ è dispari, un nodo ad altezza h' avrà solo un figlio, $n_{h'} = \lfloor n_{h'-1}/2 \rfloor + 1 = \lceil n_{h'-1}/2 \rceil$. Quindi:

$$n_{h'} = \left\lceil \frac{n_{h'-1}}{2} \right\rceil \leq \left\lceil \frac{1}{2} \left\lceil \frac{n}{2^{h'-1+1}} \right\rceil \right\rceil = \left\lceil \frac{n}{2^{h'+1}} \right\rceil$$

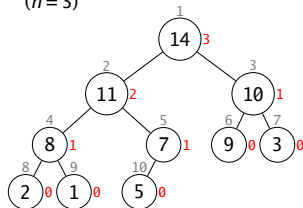
$n_{h'}$ h' ($h=3$)

$$\leq \left\lceil \frac{10}{16} \right\rceil = 1 \quad 3$$

$$\leq \left\lceil \frac{10}{8} \right\rceil = 2 \quad 2$$

$$\leq \left\lceil \frac{10}{4} \right\rceil = 3 \quad 1$$

$$\leq \left\lceil \frac{10}{2} \right\rceil = 5 \quad 0$$



Il valore in rosso è l'altezza del nodo.

HEAPSORT

Heap: Costruire un heap: BUILDMAXHEAP: complessità

- MAXHEAPIFY usa tempo $O(h')$ per ciascuno nodo ad altezza h' .
- Altezza h' varia da 1 (le foglie sono escluse) a $\lfloor \lg n \rfloor$ (radice).
- Quindi la complessità di BUILDMAXHEAP è:

$$\begin{aligned}\sum_{h'=1}^{\lfloor \lg n \rfloor} \left\lceil \frac{n}{2^{h'+1}} \right\rceil O(h') &= \sum_{h'=0}^{\lfloor \lg n \rfloor - 1} \left\lceil \frac{n}{2^{h'}} \right\rceil O(h') \leq n \sum_{h'=0}^{\lfloor \lg n \rfloor} \left\lceil \frac{1}{2^{h'}} \right\rceil O(h') \\ &= O\left(n \sum_{h'=0}^{\lfloor \lg n \rfloor} \frac{h'}{2^{h'}}\right) = O\left(n \frac{1/2}{(1-1/2)^2}\right) \\ &= O(n)\end{aligned}$$

* $\sum_{h'=0}^{\lfloor \lg n \rfloor} \frac{h'}{2^{h'}} \leq \frac{1}{2} \sum_{h'=0}^{\infty} h' \left(\frac{1}{2}\right)^{h'-1} = \frac{1}{2} \frac{1}{(1-1/2)^2} = 2$ derivata e moltiplicazione per 1/2 della serie geometrica di ragione 1/2.

HEAPSORT

Algoritmo

- Prima costruisce un max-heap.
- Poi, a partire dalla radice, scambia il primo elemento (massimo) con l'ultimo.
- La nuova radice potrebbe non essere il massimo. Quindi una chiamata a `MAXHEAPIFY(A, 1)` ripristina la proprietà di max-heap in $A[1 : n - 1]$. Si esclude l'elemento di indice n perché già nella posizione corretta.
- Itera il processo fino a quando considera l'elemento di indice 2.

Algoritmo: HEAPSORT(A)

Input: A array di n elementi

Output: A ordinato

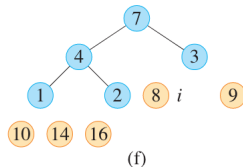
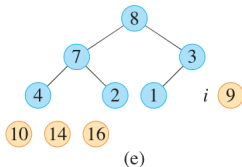
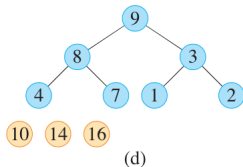
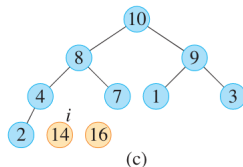
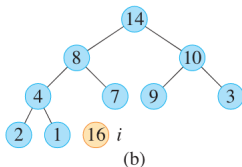
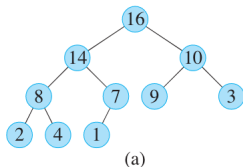
```
1: BUILDMAXHEAP( $A$ )
2: for  $i = A.length$  to 2 do
3:   | Scambia  $A[1]$  con  $A[i]$ 
4:   |  $A.heapSize =$ 
5:   |    $A.heapSize - 1$ 
   |   MAXHEAPIFY( $A, 1$ )
```

HEAPSORT

Esempio

Array in input $A = [16, 14, 10, 8, 7, 9, 3, 2, 4, 1]$

I nodi senza archi rappresentano la parte finale dell'array A .

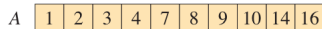
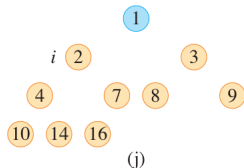
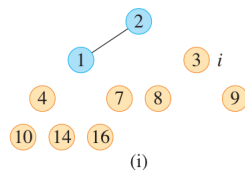
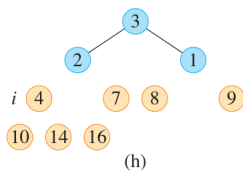
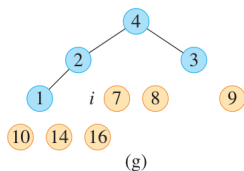


HEAPSORT

Esempio

Array in input $A = [16, 14, 10, 8, 7, 9, 3, 2, 4, 1]$

I nodi senza archi rappresentano la parte finale dell'array A .



(k)

HEAPSORT

Algoritmo: correttezza e complessità

Correttezza

È sufficiente considerare la seguente invariante di ciclo: *All'inizio di ogni iterazione del ciclo **for**, il sottoarray $A[1 : i]$ è un max heap che contiene gli i elementi più piccoli di $A[1 : n]$ e il sottoarray $A[i + 1 : n]$ contiene gli $(n - i)$ elementi più grandi di $A[1 : n]$, ordinati.*

Complessità

- La chiamata a BUILDMAXHEAP costa $O(n)$;
- Poi ci sono $n - 1$ chiamate a MAXHEAPIFY, ciascuna delle quali costa $O(\lg n)$.
- Complessità $O(n \lg n)$.

Code di priorità

Altra applicazione della struttura dati heap

Definizione 3 (Coda di max-priorità)

Una **coda di max-priorità** è una struttura dati che gestisce un insieme S di elementi, ciascuno associato a un valore detto **chiave o priorità**, in modo da estrarre l'elemento di priorità massima o di cambiare la priorità di un elemento nel modo più efficiente possibile.

Analogamente si definisce la *coda di min-priorità*.

Code di priorità

Operazioni fondamentali

Se x è un elemento, $x.key$ è la sua chiave/priorità.

Le operazioni fondamentali per le code di priorità sono:

- $INSERT(S, x)$: inserisce l'elemento x in S .
- $MAX(S)$: ritorna l'elemento di S con priorità massima.
- $EXTRACTMAX(S)$: rimuove e ritorna l'elemento di S con priorità massima.
- $INCREASEKEY(S, x, k)$: aumenta la chiave di x a k (k deve essere maggiore del valore corrente $x.key$).

Notare che se si vuole diminuire il valore di un elemento, è sufficiente farlo direttamente e poi invocare $MAXHEAPIFY$.

Analogamente si definiscono le operazioni fondamentali per le *code di min-priorità*.

Code di priorità

Operazioni fondamentali: algoritmi

Algoritmo: MAX(S)

Input: S coda di max priorità

Output: Elemento x con massima priorità

1: **return** $S[1]$

Algoritmo: EXTRACTMAX(S)

Input: S coda di max priorità

Output: Elemento x con massima priorità e rimozione dello stesso

1: **if** $S.heapSize < 1$ **then return** “Coda vuota!”

2: $max = S[1]$

3: $S[1] = S[S.heapSize]$

4: $S.heapSize = S.heapSize - 1$

5: MAXHEAPIFY[§]($S, 1$) // §MAXHEAPIFY modificata per usare $S[i].key$

6: **return** max

Complessità di EXTRACTMAX = $O(\lg n)$.

Code di priorità

Operazioni fondamentali: algoritmi

Algoritmo: INCREASEKEY(S, i, key)

Input: S coda di max priorità, i posizione di x , key nuovo valore per $x.key$

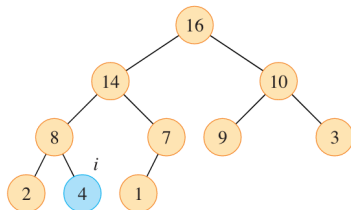
Result: Priorità $x.key$ aggiornata in S .

- 1: **if** $key < S[i].key$ **then return** “Errore. Nuova priorità è più piccola”
 - 2: $S[i].key = key$
 - 3: **while** $i > 1$ e $S[PARENT(i)].key < S[i].key$ **do**
 - 4: scambia $S[i]$ con $S[PARENT(i)]$
 - 5: $i = PARENT(i)$
-

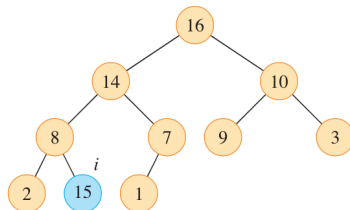
- Se la nuova key rompe la proprietà di max heap, l'elemento x viene scambiato con il genitore fino a quando non trova la sua giusta posizione.
- Complessità $O(\lg n)$ perché al più si deve spostare un elemento dalla foglia più profonda alla radice.
- Se, invece di i , si desse in input x , come si dovrebbe modificare INCREASEKEY? Quale sarebbe la nuova complessità?

INCREASEKEY

Esempio: nell'albero sono rappresentate le chiavi

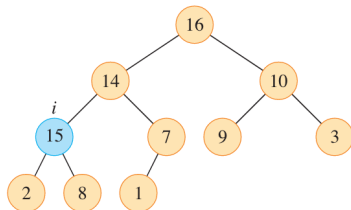


(a)

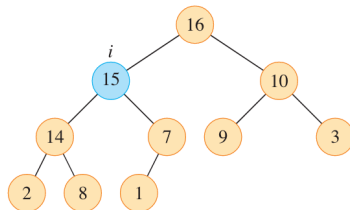


(b)

La chiave $i.key$ viene cambiata da 4, fig. (a), a 15, fig. (b).



(c)



(d)

Code di priorità

Operazioni fondamentali: algoritmi

Algoritmo: INSERT(S, x)

Input: S coda di max priorità, nuovo elemento x

Result: S aggiornato con nuovo elemento x

- 1: $S.heapSize = S.heapSize + 1$
 - 2: $key = x.key$
 - 3: $x.key = -\infty$
 - 4: $S[S.heapSize] = x$
 - 5: INCREASEKEY($S, S.heapSize, key$)
-

- Aggiunge x in fondo alla coda ponendo la chiave a $-\infty$;
- Chiama INCREASEKEY per x con il valore originale della chiave di x ;
- In questo modo x risale alla posizione corretta;
- Complessità $O(\lg n)$ perché x potrebbe essere spostato dalla foglia (più profonda) alla radice.

In questa lezione si è visto:

- La struttura dati **heap**.
- L'algoritmo HEAPSORT.
- Le code di priorità.

- Studiare cap. 6 [CLRS].
- Risolvere gli esercizi 6.1-2, 6.1-4, 6.2-3, 6.2-6, 6.3-3, 6.3-4, 6.4-3, 6.4-5, 6.5-4, 6.5-8 [CLRS].
- Risolvere il problema 6-1.
- Dimostrare che la complessità di HEAPSORT rimane $O(n \lg n)$ anche se si calcola l'esatto costo di ciascuna delle $n - 1$ chiamate a MAXHEAPIFY.